

COMP4141 Slides Week 4 + 1 ☺

Brendan Mabbutt

UNSW

March 16, 2026



UNSW
SYDNEY

UNSW
COMPUTER
SCIENCE &
ENGINEERING

Admin

Class Test Last Week

- Some students has $\mathbb{N}$ include 0 and others did not. You should use that $\mathbb{N}$ contains 0.
- This week will just be on proving a languages is not regular.

Notes from marking

- Try your construction on some examples to understand where it might fail if you are not sure (particularly related to the DN task from Week 3).
- Most who did the HD task found it easier than the DN one.

Starts on Marking Moving forwards

- This will have:
 - **0 star:** not acceptable, completely wrong.
 - **1 star:** on the right track, but not sufficient and details missing.
 - **2 stars:** almost there, but somewhat incomplete.
 - **3 starts:** all correct.

Turing Machines Formally

Scope

Church–Turing Thesis

Turing machines define computability.

Definition

Turing machine

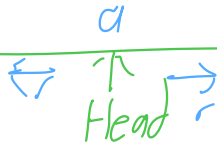
A TM is a tuple

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$$

with components:

- Q : set of states
- Σ : input alphabet, not containing the blank symbol $\sqcup$
- $\Gamma \supseteq \Sigma$: tape alphabet, containing $\sqcup$
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{l, r\}$
- q_0 : start state
- q_{accept} : accept state
- q_{reject} : reject state ($\neq q_{\text{accept}}$)

$a \in \Gamma$



→ Reach one of these then we halt

Configurations

Configuration

A configuration for a TM is a triple

$$(u, q, v) \in \Gamma^* \times Q \times \Gamma^*$$

- q is the current state.
- The tape content is uv , with the head at the first symbol of v (blanks suppressed).

For $a, b, c \in \Gamma$, $u, v \in \Gamma^*$, and $q, q' \in Q$:

- Configuration $uaqbv$ yields $uq'acv$ if $\delta(q, b) = (q', c, l)$.
- Configuration $uaqbv$ yields $uacq'v$ if $\delta(q, b) = (q', c, r)$.

Acceptance in Turing Machines

Turing machine acceptance

- $q_0 w$ is called the start configuration.
- $uq_{\text{accept}}v$ are accepting configurations.
- $uq_{\text{reject}}v$ are rejecting configurations.

A TM $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ accepts input $w \in \Sigma^*$ if a sequence of configurations $C_1, \dots, C_n$ exists such that:

1. $C_1 = q_0 w$
2. each C_i yields C_{i+1}
3. C_n is an accepting configuration

The language $L(M)$ of M is the set of strings accepted by M .

Encodings

Encodings

$\langle M \rangle$
(angle M angle)

Encodings

The input to a Turing machine is always a string, but it can be useful to work with objects other than words over Σ .

If O is a finitely presented object, an encoding of O (over Σ), written $\langle O \rangle$, is a representation of O as a string.

Universal Turing Machine

Simulating a Turing machine

A universal TM is a TM U that can simulate the actions of any Turing machine.

For any TM M and input x , U accepts (respectively rejects or loops on) $\langle M, x \rangle$ if and only if M accepts (respectively rejects or loops on) x .

This is analogous to an interpreter for C written in C.

Decidability and Recognisability

Recognisability

Turing-recognisable

A language $L \subseteq \Sigma^*$ is Turing-recognisable (or recursively enumerable, semi-decidable) if some TM M recognises it, i.e. $L = L(M)$.

If w is in L then M will
accept

If w is not in L it may reject or it
loop forever

Decidable

A TM can fail to accept some input in two ways:

- rejecting: eventually entering q_{reject}
- diverging: entering an infinite loop

Decidable

A language $L \subseteq \Sigma^*$ is (Turing-)decidable (or recursive) if some TM decides it.

↓
The TM will eventually halt

Enumerators

An enumerator is a TM-like construct that starts with a blank input tape and, if it does not halt, prints an infinite list of strings onto the tape.

It does so by occasionally entering a print state, which outputs the terminal string to the right of the head (including the symbol under the head).

Recognisability

A language is Turing-recognisable iff some enumerator enumerates it.

Recursively Enumerable vs Countable

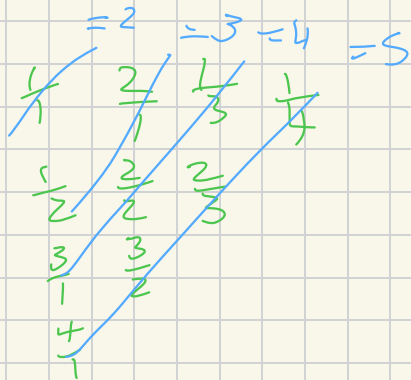
Countability

A set is said to be countable if its elements can be listed out (formally, a countable set is either finite or countably infinite; the latter meaning there is a one-to-one correspondence between it and the natural numbers $\mathbb{N}$).

Recursive enumerability for a language is stronger than countability, as it essentially asks us to compute such a list as well.

Example: $\mathbb{N}$ is countable, $\mathbb{Z}$ is countable

$\mathbb{Q}$ is countable (set of rational numbers)



Summary

L is RE (or semi-decidable) if there is a TM such that:

- For every $w \in L$, the TM accepts w in finite time.
- For every $w \notin L$, the TM either rejects or diverges.

L is recursive (or decidable) if there is a TM such that:

- For every $w \in L$, the TM accepts w in finite time.
- For every $w \notin L$, the TM rejects w in finite time.

W5 P1

Problem 1

Give a detailed description of a Turing machine that decides the language L over alphabet $\{0, 1\}$ consisting of all strings with the same number of 0's as 1's. You may find the simulator [here](#) useful to debug your solution.

0011 ∈ L

011 ≠ L

Have TM M

"On input w

1. Scan to the left until we find
a 0 or 1

a) If we find a 0 we keep scanning
until we find a 1

b) If we find a 1 we scan until 0

2. If a) or b) fail, reject, other replace the
0 and 1 with a' or (return to the

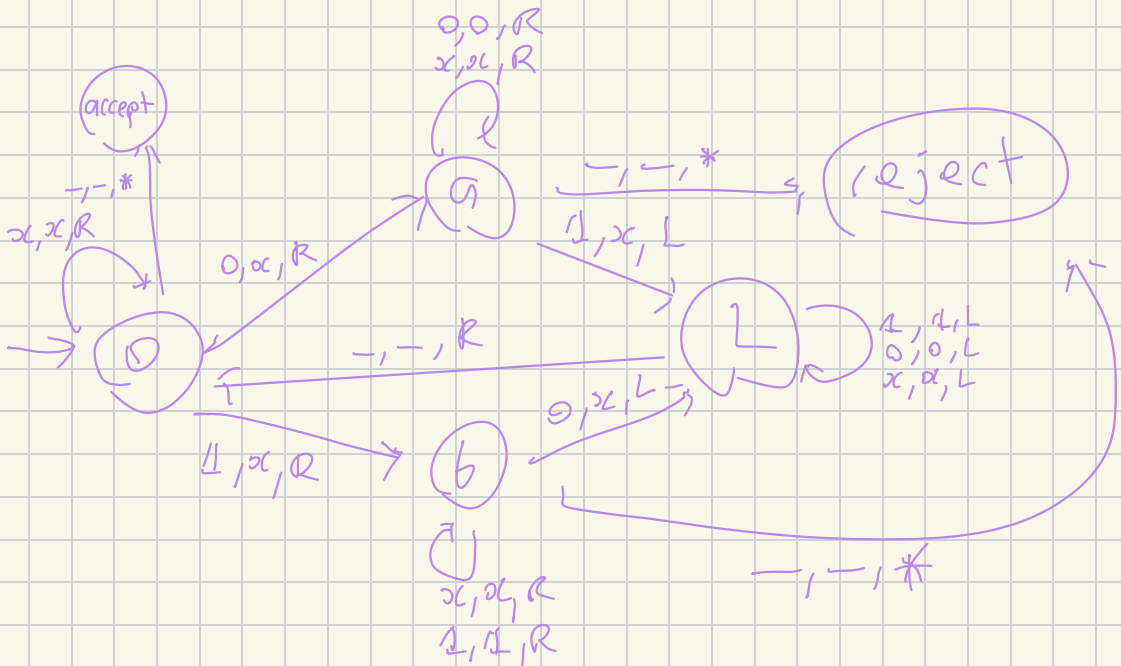
3. Repeat 1 and 2 until there is no 0 or 1; accept

$Q = \{0, a, b, L, \text{accept}, \text{reject}\}$

$q_0 = 0$ (starting state) $\rightarrow$ also where we will look for first 0 or 1

$a \rightarrow$ trying to find a 1

$b \rightarrow$ trying to find a 0



W5 P2

Problem 2

Let

$$L = \{ \langle M \rangle \mid M \text{ is a DFA that doesn't accept any string containing an odd number of 1's} \}$$

Show that L is decidable.

Let O be a DFA that accepts only (and all) strings with an odd number of 1's

$L(M) \cap L(O)$ is the set in $L(M)$ that has an odd number of 1's

then $L(m) \cap L(o) = \emptyset$ iff

$L(m)$ has no
string an odd
number of 1's

Build TM I to accept L

↳ On input $\langle M \rangle$

1) Take the M and $\emptyset$ and build a DFA B that accepts $L(M) \cap L(\emptyset)$

2) We do BFS in B to see if it can accept state $(L(M) \cap L(\emptyset) \text{ is empty})$ (test it)

3) Accept if we can reach, reject otherwise

W5 P3

Problem 3

A nondeterministic Turing machine is a *decider* for a language L if for all inputs w , the every branch of the computation tree of the machine terminates, and $w \in L$ iff at least one branch accepts. Show that L is decided by a non-deterministic Turing machine iff L is decided by a deterministic Turing machine.

($\Rightarrow$) If non-deterministic M accepts L ,
for deterministic M' , run M on
every branch and accepts if it does,
otherwise reject. This will always
terminate since every branch terminates
(There can only be a finite number of branches)

($\Leftarrow$) If deterministic M accepts L ,
 M is also non-deterministic so we are
done.

W5 P4

Problem 4

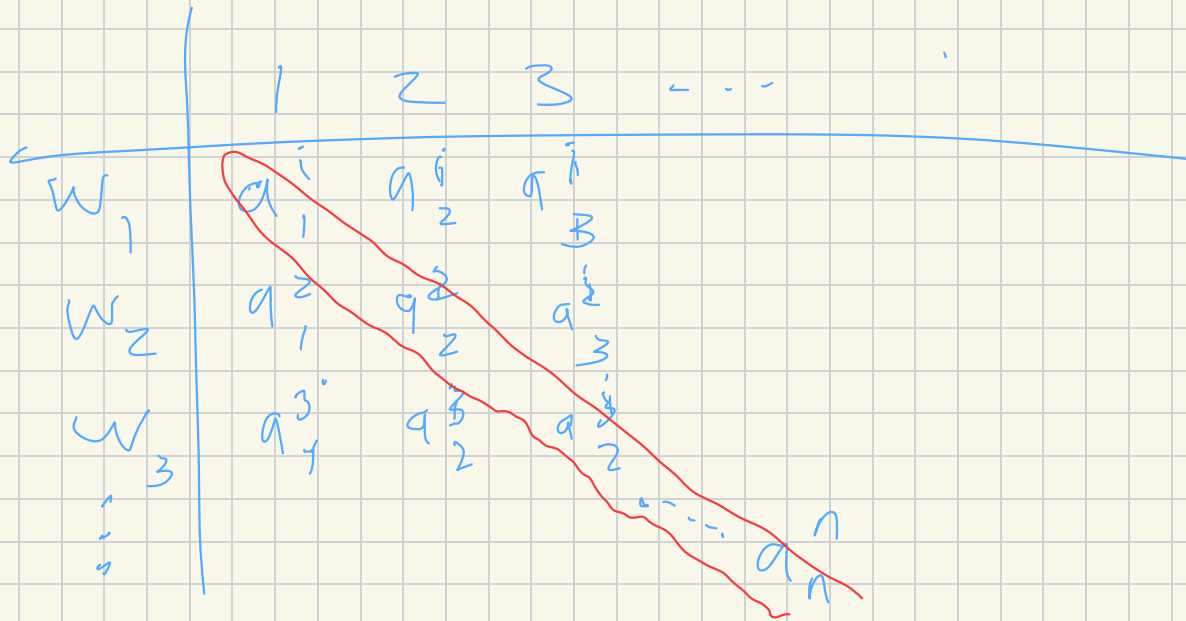
An *infinite word* over Σ is an infinite sequence $\rho = a_0 a_1 a_2 \dots$ such that $a_i \in \Sigma$ for all i . Show that the set of infinite words over $\{0, 1\}$ is uncountable.

We proceed similarly to how it was proven $[0, 1]$ was not countable

We prove assuming the set of infinite words is countable. So we can identify this $w_1, w_2, \dots$

Say that

$$w_i = a_1^i a_2^i a_3^i \dots$$



We make $w' = a'_1 a'_2 \dots$

$$a'_j = \begin{cases} 1 & \text{if } a_j = 0 \\ 0 & \text{if } a_j = 1 \end{cases}$$

So $\forall w_k, w_k$ and w' will have a different k th bit

So w' cannot be in $w_1, w_2 \dots$

but w' is a infinite word
but not counted by sequence
we cannot have such a sequence

W5 P5

Problem 5

Show that the language $\{\langle M \rangle \mid M \text{ is a Turing machine with } L(M) \neq \emptyset\}$ is Turing-recognisable.

One idea is run M on every string and accepts it M accepts

↳ in lexicographic order, at
 $w \in \Sigma^*$ run w on M

This does not work if M loops
on a word w' and we haven't found a word accepted

Number of (Words in Σ^*)

step

1

2

3

$\vdots$

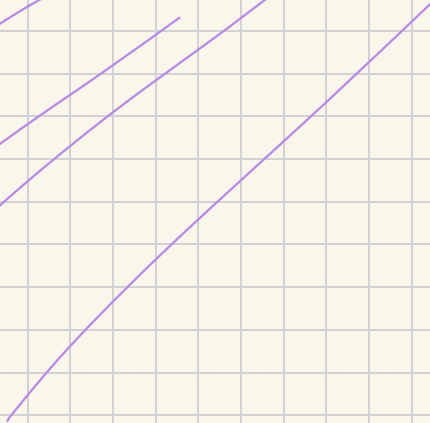
w_1

w_2

w_3

...

0



Lets set $w_1, w_2, \dots$ as the strings
of TM will

"On input $\langle M \rangle$:

1. For i from $1 \dots \infty$:

— Run $w_1, w_2, \dots w_i$ on
 M for i step

— If we have halt - accept
in any, accept (our tm)

IF $M \in L$ then $\exists w_j$
s.t. $w_j = L(M)$

and suppose we accept in
 k steps. Then in the

$\max \{k, j\}$ iteration we
will accept

Feedback

Please let me know how I can improve for next week [here!](#)

COMP4141 26T1 Feedback Form
(For Brendan Mabbutt's Tutorials)

